

A2UI 实战 (一) - 渲染器基础

👤 衿言 | 3月22日修改

🚀 AI 速览 [试用](#)

本文讨论了 A2UI 实战 (一) 中渲染器基础的相关内容，涵盖目标、环境准备、核心模块设计、初始化流程、测试策略和跨...

A2UI 实战 (一) - 渲染器基础

本系列实战将带你从零实现一个完整的 A2UI 渲染器 SDK。

前置知识: 已完成第四部分前 4 章学习, 理解 A2UI 协议、Parser、TreeBuilder、Renderer 核心概念。



a2ui-full.tar.gz

39.30MB



板书:

[📖 板书](#)

一、目标

核心目标

通过与 AI 协作的方式, 从零完成 A2UI SDK 的 MVP (最小可行产品) 版本。

- 🤖 AI 的角色: 代码生成助手、代码审查员
- 👤 你的角色: 架构设计者、需求定义者、最终决策者
- 📦 产出物: 一个可运行、可测试、可扩展的 A2UI SDK

MVP 范围

本章节 (实战一) 完成:

- @a2ui/core - JSONL Parser、TreeBuilder、Store
- @a2ui/react - 基础组件 (Text、Button、Column、Row) 、Renderer
- Web 示例 - 静态 JSONL 渲染

二、环境和准备

2.1 系统要求

- Node.js: 24+ 版本
- 包管理器: pnpm (推荐 8.x+)
- 代码编辑器: VS Code / Trae / 任意支持 TypeScript 的编辑器

检查环境:

```
代码块
1 node -v # 应 >= v24.0.0
2 pnpm -v # 应 >= 8.0.0
```

如 Node 版本不足, 推荐使用 nvm 管理

A2UI官方协议



specification_v0.8.zip

140.83KB



三、a2ui-core 详细设计

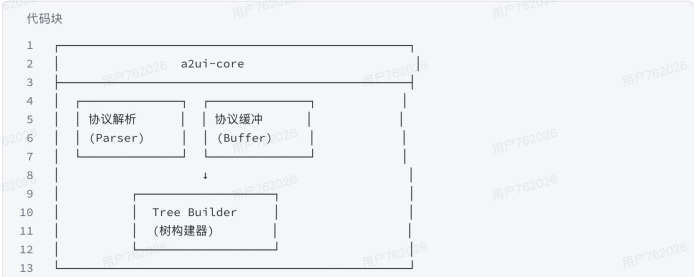
3.1 设计目标

a2ui-core 是 A2UI SDK 的核心层, 纯 TypeScript 实现, 不包含任何框架相关代码。

设计原则:

- ✅ 框架无关: 不依赖 React/Vue/Angular, 可运行在任何 JS 环境
- ✅ 跨端移植: 支持 Web、Node.js、React Native、小程序等环境
- ✅ 类型安全: 完整的 TypeScript 类型定义
- ✅ 流式友好: 支持增量解析和缓冲

核心模块:



3.2 协议解析 (Parser)

职责：将 JSONL 流解析为结构化的 A2UI 消息。

关键定义：

A2UI 消息类型：

- `CreateSurface` - 创建 Surface 容器
- `SurfaceUpdate` - 更新组件树
- `BeginRendering` - 开始渲染
- `DeleteSurface` - 删除 Surface

Parser 接口：

代码块

```
1 interface A2UIParser {
2   // 解析单行 JSONL
3   parseLine(line: string): ParseResult
4
5   // 流式解析 chunk
6   parseChunk(chunk: string): ParseResult[]
7
8   // 重置 parser
9   reset(): void
10 }
11
```

ParseResult 结构：

代码块

```
1 interface ParseResult {
2   type: 'createSurface' | 'surfaceUpdate' | 'beginRendering' | 'deleteSurface' | 'error'
3   message?: A2UIMessage
4   error?: string
5 }
6
```

设计要点：

- 单行解析：用于调试和测试
- 流式解析：支持 SSE 增量输入
- 容错处理：无效 JSON 不中断整体流程
- 状态管理：buffer 缓存不完整的行

3.3 Store 设计 (Zustand 模式)

使用 Zustand 模式的纯 JavaScript 实现，不依赖任何框架。

职责：

- 统一管理组件定义 (componentMap)
- 统一管理渲染后的 VNode 实例 (hydrateNodeMap)
- 维护组件之间的关系 (父子引用)
- 错误管理

Store 接口定义

代码块

```
1 interface A2UIStore {
2
3   // 原始协议
4   JSONLs
5
6   // 维护hydrateNode
7   hydrateNodeMap
8
9   // 获取 hydrateNodeMap
10  getHydrateNodeMap(): Map<string, VNode>
11
12  // 设置 hydrateNodeMap
13  setHydrateNodeMap(map: Map<string, VNode>): void
14
15  // 按照ID更新
16  updateHydrateNodeById,
17
18  // 错误信息
19  Error
20 }
21
```

hydrateNodeMap 结构

作用：存储渲染后的 VNode 实例，维护组件关系。

代码块

```
1 interface VNode {
2   id: string // 组件唯一标识
3   type: string // 组件类型
4   props: Record<string, any> // 组件属性
5 }
```

```
5   children: string[]           // 子组件 ID 列表 (关系引用)
6   parentId?: string           // 父组件 ID (关系引用)
7   surfaceId: string           // 所属 Surface
8   isMounted: boolean          // 是否已挂载
9   _vnode:                     // 指向已被react渲染的组件实例
10  }
11
```

hydrateNodeMap 示例:

```
代码块
1  Map {
2    "root" => {
3      id: "root",
4      type: "Column",
5      props: { spacing: "medium" },
6      children: ["title", "content"],
7      parentId: undefined,
8      surfaceId: "main",
9      isMounted: true,
10     _vnode,
11   },
12   "title" => {
13     id: "title",
14     type: "Text",
15     props: { text: "天气卡片", usageHint: "h1" },
16     children: [],
17     parentId: "root",
18     surfaceId: "main",
19     isMounted: true,
20     _vnode,
21   },
22   "content" => {
23     id: "content",
24     type: "Text",
25     props: { text: "北京今天晴, 15°C", usageHint: "body" },
26     children: [],
27     parentId: "root",
28     surfaceId: "main",
29     isMounted: true,
30     _vnode
31   }
32 }
33
```

组件关系维护

关系存储:

```
代码块
1  interface ComponentRelations {
2    // 父 → 子 关系
3    parentToChildren: Map<string, string[]>
4
5    // 子 → 父 关系
6    childToParent: Map<string, string>
7
8    // 根节点列表
9    roots: Set<string>
10  }
11
```

关系更新流程:

```
代码块
1  1. Parser 解析 SurfaceUpdate 消息
2    ↓
3  2. 提取 components 数组
4    ↓
5  3. 遍历组件, 建立关系:
6     - 如果组件有 children → 记录 parentToChildren
7     - 对于每个 child → 记录 childToParent
8    ↓
9  4. 更新 hydrateNodeMap:
10     - 创建/更新 VNode
11     - 设置 parentId 和 children
12    ↓
13  5. 通知订阅者 (Store 状态变化)
14
```

Zustand 模式实现要点

1. 状态集中管理

```
代码块
1  interface StoreState {
2    componentMap: Map<string, ComponentDef>
3    hydrateNodeMap: Map<string, VNode>
4    relations: ComponentRelations
5    newComponentIds: Set<string>
6    listeners: Set<() => void>
7  }
8
```

Parser与 Store的协作流程

